

World 8 / Z0-A - سند جامع و آرشیوی معماری کامل

Runtime پایدار مستقل از Provider با Observation مستقل و Development Proposal-only

وضعیت: FINAL DESIGN BASELINE / NOT PRODUCTION / ARCHIVAL MASTER

تاریخ تثبیت: 24-08-2026

هدف آرشیوی: این نسخه برای بازسازی پروژه در آینده نوشته شده و عمداً از نسخه کوتاه مفصل تر است. متن استخراج شده کامل از Baseline نهایی، Roadmap و Cross-Review Closure نیز در پیوست ها بدون حذف نگه داشته شده است.

1. وضعیت معماری و مرز ادعا

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Z0-A خط مبنای طراحی تثبیت شده است، نه گواهی آمادگی Production. هر ادعای اجرایی فقط با Evidence مخصوص همان Claim ارتقا پیدا می کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `design freeze is separate from deployment`
- `no autonomous-evolution claim`
- `no biological or consciousness claim`
- `evidence status remains independently versioned`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache، Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

2. حکم نهایی معماری

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

منبع حقیقت باید خارج از Cognition قابل تعویض، Session ارائه‌دهنده، Interface، Telemetry و implementation و Development Agent باقی بماند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `accepted change moves known canonical state to new canonical state`
- `explicit authority`
- `commit-time CAS`
- `intent-bound idempotency`
- `append-oriented accepted history`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

3. توپولوژی ثابت پنج Plane

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

و Canonical Spine, Operational, Observation, Development/Mason توپولوژی هنجاری دقیقاً شامل Evidence/Governance است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `Cognition is an Operational service`

- `Ingress is an Operational service`
- `Observation is not canonical truth`
- `Mason is not a promotion authority`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

4. مالکیت Canonical Spine

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Spine مالک Identity پاسخ گو، Event History پذیرفته شده، Head و Revision کانیکال، Checkpoint reference, Authorization و تاریخچه Activation است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `models cannot write directly`
- `telemetry cannot rewrite state`
- `commits require governed interface`
- `history corrections are new events`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version, Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

5. Operational Plane

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Task, Role, RoleBinding, Cognition routing، Operational سرویس های کار واقعی را انجام می دهند Context loading, Connector, Outbox dispatch و Effect settlement.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند

Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `provider identifiers are provenance`
- `workflow engines are not source of truth`
- `task state must survive process loss`
- `effects remain explicit obligations`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

6. Observation Plane

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Observation تلهمتری خام را می گیرد و به صورت مستقل Measurement، SLO result، Invariant result و gap_signal قابل انتساب تولید می کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `loss of telemetry must not break replay`
- `detector version recorded`
- `windowing recorded`
- `Mason cannot choose its own evidence window`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

7. Development / Mason Plane

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Mason یک قابلیت توسعه Proposal-only است که Gap معتبر را مصرف و Candidate Phenotype Revision بازگشت‌پذیر تولید می‌کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `cannot alter objectives`
- `cannot alter hard constraints`
- `cannot alter authority or identity`
- `cannot self-evaluate`
- `cannot self-promote`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State stale, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، می شوند و از Mock محلی استنتاج نمی شوند.

8. Evidence / Governance Plane

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Evidence/Governance مالک Predicate, Falsifier, Evidence Reference, Evaluation Receipt, Approval Policy, Promotion Gate و Audit است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `PASS does not propagate to unrelated claims`
- `OPEN means required evidence is absent`
- `human approval is not the only security boundary`
- `promotion is separately authorized`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version, Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

9. شیء World

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

World بالاترین Governance/Namespace boundary است، نه Session مدل یا Agent تعریف شده فقط با Prompt.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `contains constitutional invariants`
- `contains canonical identifier namespace`
- `can contain multiple entities and societies`
- `tracks architecture/runtime lineage`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

10. مدل Entity

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Entity یک شیء پایدار و پاسخ‌گو با Canonical ID ثابت، Authority Ceiling، State Head، Lifecycle، Event و Lineage است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `entity survives provider replacement`
- `entity is not a role name`
- `entity can be individual or organizational`
- `retirement does not silently recycle identity`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

11. Society و company-001

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

مستقل Resource و History, Objective, Role, Task, Policy عملیاتی با Entity/Society یک company-001 است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `society can host multiple roles`

- `society identity persists across executors`
- `role changes do not fork society identity`
- `organizational state is canonical where required`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

12. مدل Role

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Role جایگاه مسئولیت با Contract نسخه دار است؛ به صورت خودکار Entity با هویت مستقل نیست.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `examples: secretary, sales, accountant`
- `role name does not mint authority`
- `role can require skills`
- `role output contract is versioned`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

13. RoleBinding

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

RoleBinding اتصال Governed است که Role را برای Holder/Executor در Scope و Grant محدود فعال می کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند

Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `provider-neutral binding`
- `scope and validity explicit`
- `authority grant bounded by ceilings`
- `binding changes are auditable`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

14. Holder و Executor

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

deterministic، سرویس LLM مبتنی بر Runtime، Human Proxy جاری می تواند Holder/Executor باشد. Shared Runtime یا Dedicated Entity.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `holder is operational provenance`
- `holder replacement need not replace entity`
- `holder capabilities remain bounded`
- `credentials are scoped`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

15. Task و Task Bus

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Task واحد کار Governed با ID, State, Role/Scope, Artifact, Approval, Output و Provenance است؛ Task Bus پایدار Handoff را هماهنگ می‌کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `task may cross providers`
- `workflow engine state is not authoritative`
- `retries preserve task identity`
- `terminal/waiting states are explicit`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، می شوند و از Mock محلی استنتاج نمی شوند.

16. مدل Skill

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Skill یک Capability Contract قابل استفاده مجدد است و تا زمانی که Identity/Authority/State/Lifecycle مستقل لازم نباشد در Library/Runtime می ماند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `skills do not mint authority`
- `repeated behavior may become deterministic skill`
- `role may bind skills`
- `entity birth is a separate governance decision`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی شوند.

17. هویت مستقل از Provider

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Provider، Model، Session، Thread و Channel فقط Provenance هستند؛ Canonical Identity نباید از آن ها مشتق شود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `provider handoff preserves World/Entity/Role/Task IDs`
- `provider switch may be recorded as provenance`
- `identity change requires canonical event`
- `session closure is not identity death`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

18. Objective Contract

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Goal و Boundary سخت در Objective Contract کانیکال، Immutable و Versioned ذخیره می‌شوند، نه فقط داخل Prompt.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `objective`
- `success metrics and SLOs`
- `hard constraints`
- `authority ceiling`
- `risk class`

- `observation contract`
- `allowed adaptation surface`
- `promotion policy`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

مرجع Objective Contract Schema

```
{
  "objective_contract_id": "obj-...",
  "version": 1,
  "scope": "world|entity|role|task",
  "objective": "...",
  "success_metrics_and_slos": [],
  "hard_constraints": [],
  "authority_ceiling": [],
  "risk_class": "...",
  "observation_contract": {},
  "allowed_adaptation_surface": [],
  "promotion_policy": {},
  "created_by": "...",
  "created_at": "...",
  "previous_version": null
}
```

Phenotype Revision Schema [مرجع](#)

```
{
  "phenotype_revision_id": "rev-...",
  "parent_revision_id": "rev-...",
  "objective_contract_id": "obj-...",
  "objective_version": 1,
  "change_set": [],
  "migration_plan": {},
  "rollback_plan": {},
  "expected_metrics": [],
  "risk": {},
  "evidence_requirements": [],
  "proposal_origin": "human|mason|other",
  "status": "CANDIDATE"
}
```

Canonical Event Schema [مرجع](#)

```
{
  "event_id": "evt-...",
  "world_id": "world-001",
  "entity_id": "company-001",
  "event_type": "...",
  "event_version": 1,
  "occurred_at": "...",
  "committed_at": "...",
  "actor_principal": "...",
  "authority_ref": "...",
  "task_id": "...",
  "objective_ref": {"id": "obj-...", "version": 1},
  "expected_head": "...",
  "previous_head": "...",
  "idempotency_key": "...",
  "intent_hash": "...",
  "payload": {},
  "payload_hash": "...",
  "event_hash": "...",
  "fencing_token": 42,
  "evidence_refs": []
}
```

Gap Signal Schema [مرجع](#)

```
{
  "gap_id": "gap-...",
  "objective_ref": {"id": "obj-...", "version": 1},
  "metric_or_invariant_id": "...",
  "window": {"start": "...", "end": "..."},
  "detector_version": "...",
  "observed_value": "...",
  "expected_condition": "...",
  "severity": "...",
  "telemetry_refs": [],
  "provenance": {},
  "status": "OPEN"
}
```

Evaluation Receipt Schema مرجع

```
{
  "evaluation_receipt_id": "eval-...",
  "evaluator_principal_id": "...",
  "evaluator_role_or_credential_ref": "...",
  "evaluator_build_digest": "...",
  "candidate_revision_id": "...",
  "objective_ref": {"id": "obj-...", "version": 1},
  "test_suite_digest": "...",
  "environment_digest": "...",
  "result": "PASS|FAIL|INCONCLUSIVE",
  "metrics": {},
  "evidence_refs": [],
  "evaluated_at": "...",
  "attestation_or_signature": null
}
```

19. مسیر تغییر قانون اساسی

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Constitutional invariant یا Objective, Hard Constraint, Authority Ceiling, Identity Rule تغییر نیست Mason معمولی Optimization یک

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می‌شوند:

- `separate governance authorization`
- `stronger evidence/approval`
- `versioned change`
- `rollback/transition plan`
- `audit trail`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده

می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی شوند.

20. Phenotype Revision

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Phenotype Revision تغییر Candidate اجرایی/Configuration نسخه دار است که به Parent Revision و Objective Version مشخص متصل است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `change set`
- `migration plan`
- `rollback plan`

- `expected metrics`
- `risk`
- `evidence requirements`
- `candidate status before activation`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، می شوند و از Mock محلی استنتاج نمی شوند.

21. Authority جبر

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Task Grant، فعال Entity Ceiling، RoleBinding Grant بین Intersection حاصل Effective Authority است. Temporal/Scope validity و Policy Allowance، احتمالی.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow

Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `edges are conditions not grants`
- `role names do not grant`
- `authority expansion is privileged`
- `fail closed on ambiguity`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون State stale، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

22. Authorization رکورد

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

هر Canonical Change یا External Effect حساس باید به Authorization Record یا Authority Context قابل استنتاج متصل باشد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `principal identity`
- `scope`
- `required capability`
- `grant source`
- `expiry`
- `risk class`
- `policy decision`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون State stale، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Artifact Reference و Environment/Build Identity در E3 و بالاتر،

Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می‌شوند و از Mock محلی استنتاج نمی‌شوند.

23. Schema رویداد کانیکال

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

شرط، Canonical Event، Lineage، Authority، Objective/Revision context، محتوای معنایی، می‌کند bind را Provenance و Concurrency.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `event\_id unique`
- `event\_version explicit`
- `expected/previous head`
- `intent hash`
- `payload hash`
- `fencing token`
- `evidence refs`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval.

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

24. Commit-time CAS

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Expected Head باید داخل همان Transaction بررسی شود که Canonical Head را Advance می‌کند؛ Pre-read در Application کافی نیست.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `two writers one head; only one commits`
- `stale writer fails closed`
- `no silent overwrite`
- `retry requires new proposal or rebase`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

25. Intent متصل به Idempotency

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Idempotency Key به Semantic Intent bind است؛ Retry یکسان Deduplicate می‌شود ولی استفاده همان Key برای Intent متفاوت Collision است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `durable idempotency record`
- `semantic hash`
- `target/type included`
- `collision is explicit`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

26. Transition کانیکال اتمیک

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Transition برای Outbox Obligation ضروری و ایجاد Event Append, Head Advance, Projection هستند Atomic coupled به صورت Governed.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `fault injection must not create half-state`

- `transaction rollback on partial failure`
- `analytical projections may be asynchronous`
- `authoritative minimum remains atomic`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

27. Transactional Outbox

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

External Effect به صورت Obligation ثبت شده همراه Intent کانیکال مدل می شود و بعد Effect Executor آن را claim می کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Telemetry و Engine, Model Call, UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `obligation has stable id`
- `attempts audited`
- `provider id captured`
- `ambiguous outcome reconciled`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version, Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

28. Effect Receipt

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

موفقیت یا Settlement اثر بیرونی با Receipt ثبت می شود که Obligation, Attempt, Connector, Provider result و Reconciliation state را متصل می کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند

Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `scoped effectively-once only`
- `no universal exactly-once claim`
- `duplicate protection connector-specific`
- `irreversible ambiguity is first-class`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

29. Sequencer Lease

وضعیت: NORMATIVE مگر جایی که صریحاً برجسب دیگری خورده باشد.

Sequencer Lease Authority جاری برای Serialization یک Scope را مشخص می کند و Expiry/Renewal صریح دارد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `lease state durable for claim scope`
- `single active authority or equivalent serialization`
- `expired holder cannot be trusted by itself`
- `monitor lease health`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون State stale، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

30. Fencing Token

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

هر Sequencer Epoch یک Fencing Token افزایشی می‌گیرد تا Process قدیمی حتی اگر زنده بماند Reject شود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `storage-side validation`
- `stale token rejection`
- `fresh token after failover`
- `fresh higher token after restore`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State stale, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر،

Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می‌شوند و از Mock محلی استنتاج نمی‌شوند.

31. Append-only در سطح DB

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

History متعهدشده با Privilege/Control پایگاه داده محافظت می‌شود، نه فقط Convention کد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `runtime roles denied UPDATE`
- `denied DELETE`
- `denied TRUNCATE`
- `correction is a new event`
- `break-glass admin is audited`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون State state، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

32. Canonicalization و Hash

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Semantic Hash از Serialization deterministic و Formula نسخه دار استفاده می کند تا Content و Chain قابل Verify باشد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `RFC 8785/JCS is a suitable reference`
- `hash binds previous head and semantic content`
- `hash is tamper-evidence`
- `hash is not confidentiality or authorization`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می‌شوند و از Mock محلی استنتاج نمی‌شوند.

Checkpoint مدل 33.

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Checkpoint یک State materialized متصل به Canonical Head و Reducer/Schema Version است؛ Recovery را سریع می‌کند ولی جای History پذیرفته‌شده را نمی‌گیرد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `checkpoint hash/reference`
- `version compatibility`
- `periodic verification`
- `stale checkpoints detected`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

34. Replay تایید شده

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Replay قبل از Materialization، Chain، Hash، Event Version، Duplicate ID، Ordering، Reducer Compatibility و Final Head را Verify می‌کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `corruption fails closed`
- `mutation removing hash check must be caught`
- `duplicate event id rejected`
- `unsupported version requires migration path`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

35. Clean-host Restore

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Continuity با Restart همان Process ثابت نمی شود. Recovery باید روی Clean Environment بازسازی شود و سپس Fresh Governed Write بپذیرد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `different system identity`

- `restore schema/data`
- `verify chain/head/objective/revision`
- `recreate separated credentials`
- `fresh lease/fencing`
- `record RTO/RPO and restore receipt`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

36. Ingress و Routing

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

External Input Normalize و در صورت نیاز Authenticate می شود، به Entity/Task/Role Scope resolve می شود و بدون Direct Truth Mutation Route می گردد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow

Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `channel-specific data stays provenance`
- `routing decision auditable`
- `unknown identity handled explicitly`
- `input cannot bypass policy`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State approval, Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

37. Brain Gateway

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Brain Gateway از Scoped Context, Role Contract, Tool Provider, Cognition مستقل می کند. Output Schema و Authority Limit.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `ChatGPT/Grok/DeepSeek/Gemini/Claude/local/deterministic possible`
- `provider selection by policy`
- `brain output is proposal/result`
- `brain cannot directly commit`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

38. Context Compiler L K0 تا K4

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Context: World Kernel, Entity/Society Context, Role Contract, Scope بر اساس Context, Evidence/Memory دارای Provenance و حداقل Current Task.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `minimize context without losing invariants`
- `source remains outside generated summary`
- `staleness tracked`
- `provider context window is not canonical memory`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، می شوند و از Mock محلی استنتاج نمی شوند.

39. معماری Memory

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Working, Episodic, Semantic، جداسازی و می تواند Authoritative State از Retrieval Memory، Document/Evidence، Cold Archive و Protected Vault داشته باشد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `summary does not replace source`
- `vector index is rebuildable projection`
- `provenance and timestamps retained`
- `retention/privacy policy explicit`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی شوند.

40. Observation Pipeline

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Pipeline raw telemetry -> normalization -> aggregation/windowing -> SLO/invariant evaluation -> gap_signal -> Evidence Registry است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `detector outside Mason`
- `source refs retained`
- `window and method recorded`
- `coverage gaps explicit`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

41. قرارداد SLO و Invariant

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Invariant و Success Metric به عنوان بخشی از Objective/Observation Contract نسخه دارند تا Optimization با Target پایدار سنجیده شود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می‌شوند:

- `threshold changes are versioned`
- `identity continuity can be invariant`
- `authorization-before-effect can be invariant`
- `restore success can be SLO/gate`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

42. Gap Signal

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Gap یک Discrepancy با انتساب قابل Objective Version, Metric/Invariant, Window, Detector Version, Observed Value, Expected Condition و Evidence Ref است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `status OPEN/CLOSED/INVALIDATED/SUPERSEDED/EXPIRED`

- `Mason does not own gap definition`
- `confidence/severity optional but explicit`
- `gap provenance immutable`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

43. قرارداد Proposal در Mason

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Mason به Gap معتبر و Objective جاری reference می دهد، کوچکترین Intervention reversible را پیشنهاد می کند، Test/Rollback می دهد و داخل Adaptation Surface می ماند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Telemetry و Engine, Model Call, UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `proposal has parent revision`
- `proposal has risk`
- `proposal has expected metrics`
- `forbidden fields rejected before evaluation`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version, Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

44. هویت Evaluator

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

هویت Evaluator باید به Principal احراز شده و Build/Profile دقیق قابل انتساب باشد؛ DB Constraint به تنهایی Proof کافی نیست.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند

Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `principal id`
- `credential/role ref`
- `build digest`
- `candidate/objective refs`
- `environment/test digests`
- `metrics/evidence`
- `optional attestation/signature`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون State stale، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

45. Evaluation Receipt

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Evaluation Receipt Evaluators, Candidates, Objectives, Test Environment, Result, Metric و Evidence است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می‌شوند:

- `PASS/FAIL/INCONCLUSIVE explicit`
- `receipt immutable once committed`
- `promotion references receipt`
- `receipt does not store secrets`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

46. تفکیک وظایف

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Observation اندازه‌گیری می‌کند، Mason Proposal می‌دهد، Evaluator قضاوت می‌کند، Promotion Authority ارتقا می‌دهد، Spine ثبت می‌کند و Effect Executor Obligation را settle می‌کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `separate credentials for high assurance`
- `single host may contain logical modules only at low evidence levels`
- `self-approval forbidden`
- `one actor controlling all stages violates Z0-A`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version, Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می‌شوند و از Mock محلی استنتاج نمی‌شوند.

47. Promotion Gate

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Candidate فقط پس از Evidence/Policy Check و Promotion Decision مستقل می‌تواند با CAS فعال شود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `receipt required`
- `hard constraints unchanged`
- `authority unchanged unless constitutional path`
- `stale active revision rejected`
- `activation itself is canonical event`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

48. Rollback

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Rollback یک Canonical Event جدید برای فعال کردن Known-good یا Repair Revision است و History شکست خورده را پاک نمی‌کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `trigger recorded`
- `incident/evidence refs`
- `authority recorded`
- `post-rollback verification`
- `external effect compensation handled separately`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

49. سطوح Evidence از E0 تا E5

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Evidence Level بین Definition, Static Artifact, Local Executable, Persistent Integration, Production-like Testing و Longitudinal Operation فرق می گذارد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `E0 defined`

- `E1 static`
- `E2 local/disposable`
- `E3 persistent/integration`
- `E4 restore/security/load/chaos/independent credentials`
- `E5 longitudinal SLO/failure envelope`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

50. Proof Registry

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

و Claim اصلی Predicate, Falsifier, Required Evidence Level, Status, Environment/Build Evidence Reference دارد مستقل.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow

Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `OPEN is not failure but absence of required proof`
- `runner output should drive status`
- `manual status cannot override evidence`
- `claims do not inherit neighbor PASS`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

51. Threat Model

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Provider, Credential, Concurrency, Operator, Telemetry، Failure و Adversary معماری در نظر می گیرد External Effect و Storage.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Engine، Model Call، UI و Telemetry در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `stale writer`
- `compromised model`
- `prompt injection`
- `compromised Mason`
- `evaluator substitution`
- `credential theft`
- `partial DB failure`
- `corrupt backup`
- `operator error`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache، Service decomposition یا Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

52. Secret و Credential

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

نگه داشته می شود و فقط Evaluation Receipt و Canonical Event Payload خارج از Secret Material می آید Audit Durable در Reference/Principal.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `rotation does not change identity`
- `logs scanned for leaks`
- `least privilege`
- `high-risk credentials separated`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

53. ماتریس DB Role

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Role های PostgreSQL، Sequencer، Observer، Mason، Evaluator، Effect Executor، Promotion Authority، Audit و Migration را جدا می‌کنند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می‌شوند:

- `GRANT/REVOKE versioned`
- `negative privilege tests`
- `runtime cannot rewrite events`
- `evaluator cannot promote`
- `effect executor cannot self-authorize`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده

می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version, Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

54. Service/API مرز

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Boundary شامل Commit, Query/Projection, Task, Brain Gateway, Context Compiler, Observation, Gap Detection, Mason, Evaluation, Promotion, Effect و Recovery است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکیال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Telemetry و Engine, Model Call, UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `not required to be microservices`
- `contracts matter more than process count`
- `capability boundaries explicit`

- `API versions governed`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

55. Lifecycle State

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Lifecycle Entity یک State Machine عملیاتی صریح است، نه تابع باز یا بسته بودن Session مدل.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `PROVISIONED`

- `ACTIVE`
- `DEGRADED`
- `FROZEN/QUARANTINED`
- `HIBERNATED`
- `RESTORED`
- `RETIRED/TOMBSTONED`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی شوند.

56. Health, Repair, Sleep و Wake

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

مفاهیم قبلی Health, Sleep/Wake, Repair, Freeze و Reconstruction فقط وقتی حفظ می شوند که به Evidence و Transition صریح نگاشت شوند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند

Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `sleep can reduce active cognition`
- `wake reloads current canonical state`
- `repair is governed revision/recovery`
- `no biological claim`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

57. Multi-brain Routing

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

چند Provider شناخت می توانند پشت Brain Gateway باشند و بر اساس Capability، Cost، Latency، Privacy، Jurisdiction، Availability یا Policy انتخاب شوند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `fallback preserves identity`
- `ensemble vote is not authority`
- `provider metadata retained as provenance`
- `hard constraints cannot weaken on fallback`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

58. رشد Skill به Role و Entity

وضعیت: NORMATIVE مگر جایی که صریحاً برجسب دیگری خورده باشد.

رفتار موفق تکراری ممکن است Skill deterministic شود، Specialization پایدار Role توجیه کند و نیاز به Isolation/State مستقل Entity جدید را توجیه کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `growth is evidence-driven`
- `Mason cannot autonomously birth entities in Z0-A`
- `authority ceilings inherited/bounded`
- `genome/evolution language remains exploratory`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

59. Risk Class برای Effect

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. External Action بر اساس Reversibility و Impact طبقه‌بندی می‌شود تا Authorization/Evidence متناسب با Risk افزایش یابد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `read-only`
- `reversible low-risk write`
- `externally visible communication`
- `financial/legal/contractual`
- `destructive/high-impact`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

60. مدل Audit

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Audit باید بازسازی کند چه کسی/چه چیزی، چه Action، تحت کدام Objective/Authority، روی کدام Head، با کدام Provider/Build و با چه Result انجام داده است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `link events/tasks/principals`
- `link objective/revision`
- `link effects/receipts`
- `link gaps/evaluations/promotions`
- `survives workflow engine loss`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Version, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

61. ماتریس Failure و Response

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Failure Handling صریح است و هرجا Ambiguity بتواند Identity, Authority, History یا Effect برگشت ناپذیر را خراب کند Fail-closed می شود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `stale writer -> reject`

- `collision -> reject`
- `partial transaction -> rollback`
- `telemetry outage -> coverage degraded`
- `hash mismatch -> stop replay`
- `ambiguous effect -> reconcile`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

62. Adversarial Testing

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Test ها عمدتاً Race, Stale Token, Duplicate ID, Privilege Violation, Corrupt Replay, Forbidden Mason Change و Stale Promotion می کنند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow

Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `negative tests mandatory`
- `fault injection`
- `cross-provider continuity test`
- `restore on clean host`
- `effect ambiguity test`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

63. Mutation Testing

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Mutation Gate ثابت می کند حذف Check حیاتی مثل Replay Hash Verification یا Idempotency Collision Handling باعث Fail شدن Test می شود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `avoid vacuous test counts`
- `report mutation score by family`
- `critical mutants cannot survive`
- `runner/evidence registry linked`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

64. Deployment Profile

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Persistent Integration تا Local Development از Deployment Profile به Evidence Level نگاشت می شود Longitudinal Operation جدا و Credential با Production-like.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `E2 local`
- `E3 persistent integration`
- `E4 restore/security/load/chaos`
- `E5 longitudinal`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

65. معماری Backup

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Backup شامل Canonical DB، Event History، Schema/Migration، Objective/Revision Artifact، Evidence Registry، Manifest و Protected Recovery Procedure است.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `backup is not proof until restore tested`
- `checksums and manifests`
- `restore order documented`
- `source code alone is insufficient`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

66. انضباط Release و Repository

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. Artifact هنجاری Versioned است و Stable Release immutable؛ Fix، Manifest، Version، Hash و جدید منتشر می‌شود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `no secrets/private payloads in release`
- `schema migrations versioned`
- `rollback plan for destructive change`
- `machine-readable contracts accompany prose`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

67. Lineage از World v6.2

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

خارج از World v6.2 Entity Stability، Replaceable Brain، Scoped Context، Authority Cognition، Effect Handling، Evidence Gate و Recovery Discipline را به ارث گذاشت.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow Provenance ثبت می‌شوند و نباید به Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `retained where compatible`
- `older topology not automatically current`
- `fractal/multi-brain ideas remain operational patterns`
- `Z0-A is stricter about five planes`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون State state, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference، در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

68. Lineage از World 7

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

World 7 Persistent Identity, Spine, Proposal-only Cognition, Expected-head Conflict, Idempotency, Hash Chain, Reconstruction و Proof Obligation صریح را تقویت کرد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `genomic metaphor moved out of normative core`

- `autonomous evolution claim rejected without evidence`
- `review exposed vacuous/self-referential tests`
- `mutation and registry discipline strengthened`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

69. Lineage از World 8 v0.1/v0.1.1

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

World 8 v0.1/v0.1.1 Lineage عملیاتی world-001، company-001، RoleBinding، Task Bus و Persistent Canonical History را فراهم می کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `prior evidence remains valuable`
- `new Z0-A gates require new evidence`
- `Observation/Mason split is new fixed boundary`
- `evaluator attribution is strengthened`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version, Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

70. برداشت های منسوخ

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. برداشت قدیمی متناقض فقط برای History حفظ می شود و نباید بی صدا به Current Architecture برگردد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow

Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می‌شوند:

- `more than five normative planes`
- `Role treated as Entity by name`
- `Mason-controlled telemetry/gaps`
- `direct self-promotion`
- `objective only in prompt`
- `hash chain called secure storage`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Artifact Reference و Environment/Build Identity در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می‌شوند و از Mock محلی استنتاج نمی‌شوند.

71. عدم ادعای صریح

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

اثبات شده Consciousness, Biological Life, AGI, Legal Personhood, Autonomous Evolution معماری را ادعا نمی‌کند Freeze Design صرفاً با Production Readiness یا Universal Exactly-once

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `claim modesty is normative`
- `metaphors are operational only`
- `production requires E3/E4/E5 as applicable`
- `security claims remain scoped`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

72. Safety Property های رسمی

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Claim های Core به صورت Safety Property قابل ابطال بیان می شوند تا معماری با Test سنجیده شود نه با Metaphor.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `provider replacement preserves identity`
- `stale head cannot commit`
- `stale fencing cannot commit`
- `Mason cannot mutate hard boundaries`
- `promotion requires attributed receipt`
- `restore requires fresh write`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

73. Liveness Goal

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.
System باید در نهایت Task معتبر را Resolve کند، Lease منقضی را جایگزین کند، Effect را Settle/Reconcile کند، Candidate را Decide کند، Entity را Recover کند و Gap را Close/Invalidate کند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `liveness never bypasses safety`
- `timeouts become explicit states`
- `stuck work is observable`
- `manual escalation is allowed but audited`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval،

Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property در Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

74. سناریوی End-to-End مرجع

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Proof مرجع هر پنج Plane را لمس می‌کند: Task پایدار، Effect، Governed Commit، Provider Handoff، Receipt واقعی کم‌ریسک، Evaluation، Mason Proposal، SLO Breach مستقل و Promotion جدا.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می‌شوند:

- `same entity/task identity across provider change`
- `effect authorization/outbox/receipt`
- `gap from predefined SLO`
- `revision activated with CAS`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

75. Roadmap اجرایی Z0-A

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Freeze Contract به Spine Hardening، Entity/Role، Objective/Revision، Observation، Mason، Evaluation، Promotion، Recovery و Operational Proof می‌رود.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `Z0A-0 contracts`
- `Z0A-1 spine`
- `Z0A-2 identity/role`
- `Z0A-3 objective/revision`
- `Z0A-4 observation`

- `Z0A-5 mason`
- `Z0A-6 evaluator`
- `Z0A-7 promotion`
- `Z0A-8 restore`
- `Z0A-9 operational proof`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery و Failure Injection روی Integration هدف آزمون می شوند و از Mock محلی استنتاج نمی شوند.

76. Gate های خروج Z0-A

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد. قبل از Closed شدن Gate, Architecture و Executable Boundary باید هم خوان باشند؛ Compliance فقط در Document کافی نیست.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider, Workflow

Engine, Model Call, UI و Telemetry در صورت حضور به عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می‌شوند:

- `G1 plane isolation`
- `G2 entity/role split`
- `G3 objective/revision versioning`
- `G4 observation independence`
- `G5 Mason confinement`
- `G6 evaluator identity`
- `G7 spine atomicity`
- `G8 recovery`
- `G9 effects`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می‌تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Artifact Reference و Environment/Build Identity در E3 و بالاتر، Process/Credential مستقل در صورت لزوم، Failure Injection و Recovery روی Integration هدف آزمون می‌شوند و از Mock محلی استنتاج نمی‌شوند.

77. Runbook بازسازی برای Maintainer آینده

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Maintainer آینده از Version تثبیت شده شروع می کند، Manifest/Schema، Proof/Role Matrix، Replay، Adversarial Gate و Restore Receipt را Verify می کند و بعد Development را ادامه می دهد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به عنوان Provenance ثبت می شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement های این بخش در فهرست زیر ثبت می شوند:

- `do not trust old diagrams over current manifest`
- `verify active objective/revision`
- `verify evaluator attribution`
- `verify latest restore`
- `verify provider handoff and effect reconciliation`

هرجا Ambiguity بتواند Identity، Authority، Accepted History، Objective Boundary، Revision، Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میانبرهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale، Objective change بدون State stale، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

Evidence تعهد

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection و Mock محلی استنتاج نمی‌شوند.

78. حداقل Artifact ماشین‌خوان

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Prose باید کنار Schema, Role Matrix, Forbidden Transition, Promotion Policy, Proof Registry, Runtime Module, Observation Module, Evidence و Release Manifest باشد.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider, Workflow, Telemetry و Engine, Model Call, UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `Objective schema`
- `Revision schema`
- `Event schema`
- `Gap schema`
- `Evaluation receipt schema`
- `RoleBinding schema`
- `Outbox/effect schema`
- `DB role matrix`
- `proof registry`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت‌ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده

می‌تواند Storage، Batching، Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می‌تواند تغییر دهد.

میان‌برهای ممنوع و تفسیر Failure

Happy-path موفق به‌تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی Objective change، State stale بدون Version، Self-approval، Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می‌شود. اگر Property Evidence Level ادعاشده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچک‌ترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می‌شوند و از Mock محلی استنتاج نمی‌شوند.

79. قرارداد نهایی Continuity

وضعیت: NORMATIVE مگر جایی که صریحاً برچسب دیگری خورده باشد.

Cognition و Interface، Identity قابل تعویض‌اند؛ Canonical State، Authority، Objective، Accepted History و Evidence Governed و Persistent می‌مانند.

دامنه و مالکیت

هدف این بخش این است که Ownership و Trust Boundary مبهم نماند. هر Object کانیکال یا Governed که در طول زمان تغییر می‌کند باید شناسه پایدار و Version Reference مناسب داشته باشد. State موقت Runtime می‌تواند Cache یا Projection باشد، اما حق ندارد Authoritative Object را بی‌صدا بازتعریف کند. Provider، Workflow، Telemetry و Engine، Model Call، UI در صورت حضور به‌عنوان Provenance ثبت می‌شوند و نباید به Canonical Identity ارتقا پیدا کنند.

قواعد هنجاری

حداقل Requirement‌های این بخش در فهرست زیر ثبت می‌شوند:

- `prose and runtime must agree`
- `no bypass path`
- `all stronger claims remain evidence-gated`

- `future revisions require explicit governance`

هرجا Ambiguity بتواند Identity, Authority, Accepted History, Objective Boundary, Revision Activation یا External Effect برگشت ناپذیر را تغییر دهد، رفتار باید Fail-closed باشد. Implementation آینده می تواند Storage, Batching, Cache یا Service decomposition را Optimize کند، اما Semantic Boundary را فقط با Architecture Version جدید و Governance صریح می تواند تغییر دهد.

میان برهای ممنوع و تفسیر Failure

Happy-path موفق به تنهایی کافی نیست اگر Bypass Path وجود داشته باشد. Direct Write از Cognition قابل تعویض، Silent Overwrite روی State stale, Objective change بدون Version, Self-approval, Provenance ناقص یا سرایت PASS از Claim نامرتبط Failure معماری محسوب می شود. اگر Property Evidence Level ادعا شده اثبات نشده باشد، Proof Registry باید OPEN یا FAIL بماند و متن معماری حق ارتقای مصنوعی آن را ندارد.

تعهد Evidence

Evidence باید کوچکترین Test بازتولیدپذیر را داشته باشد که Claim را بتواند Falsify کند، همراه Persistent Storage، Environment/Build Identity و Artifact Reference. در E3 و بالاتر، Recovery روی Integration هدف آزمون Process/Credential مستقل در صورت لزوم، Failure Injection می شوند و از Mock محلی استنتاج نمی شوند.

پیوست A - الگوریتم مرجع Commit

```
BEGIN TRANSACTION
  authenticate principal
  load/verify authorization context
  verify active sequencer lease
  verify fencing_token >= current_fence and belongs to active epoch
  verify expected_head == canonical_head
  validate schema/event version
  compute semantic intent hash
  check idempotency record
    if same key + same intent: return prior committed result
    if same key + different intent: IDEMPOTENCY_COLLISION
  append canonical event
  advance canonical head
  update required authoritative projections
  create outbox obligation if external effect is planned
COMMIT
```

هیچ Pre-read در Application جای Predicate داخل Transaction را نمی گیرد. هیچ خروجی Model/Provider نمی تواند Authority/Commit Rule را bypass کند.

پیوست B - ماتریس تفکیک وظایف

Capability	مجاز	ممنوع
Promotion Authority / Human Root	Gate تحت Approve/Reject	Rewrite History یا جایگزین کردن Security Control فنی
Sequencer	Commit با Lease/Fencing/CAS معتبر	Self-approve Development Change
Observer	Signal و Measurement قابل انتساب	Canonical Mutation یا Objective rewrite برای Measurement خودش
Mason	Candidate Revision Proposal	Self-evaluate، Self-promote تغییر Hard Boundary
Evaluator	Receipt و Judge Candidate	Promote همان Candidate
Effect Executor	Settle Obligation مجاز	ساخت Authorization برای خودش
Brain Provider	Cognition/Proposal/Result	Canonical Identity یا Direct مالکیت Commit

پیوست C - Negative Test های اجباری

1. دو Writer با expected head یکسان: دقیقاً یکی Commit کند.
2. Same key + same intent: Deduplicate/Replay prior result.
3. Same key + changed intent: Collision صریح.
4. Stale fencing token: Reject.
5. Fault بین Event/Head/Outbox: Half-state ایجاد نشود.
6. UPDATE/DELETE/TRUNCATE توسط Runtime Role Committed Event: Denied.
7. Replay با Payload/Hash دستکاری شده: Reject.
8. Duplicate event_id: Reject.
9. Mutation حذف Hash Verification: Test باید Fail شود.
10. Mutation حذف Idempotency Collision: Test باید Fail شود.
11. Mason برای Objective/Hard Constraint/Authority/Identity/Spine: Reject.
12. Mason برای Promotion: Reject.
13. Promotion بدون Evaluation Receipt قابل انتساب: Reject.
14. Promotion روی Stale Revision/Head: Reject.

15. Evaluator Receipt با Principal/Build یا mismatch: Gate را نیندد.
16. Effect بدون Committed Obligation: Reject.
17. Provider Timeout مبهم: Reconciliation، نه Blind Retry.
18. Clean-host Restore: Verify Chain/Revision و یک Fresh Governed Write.
19. Provider Handoff: همان World/Entity/Role/Task ID حفظ شود.
20. Telemetry Outage: Coverage degraded و Gap جعلی ساخته نشود.

پیوست D - حداقل ساختار Repository

```

/contracts/
  OBJECTIVE_CONTRACT.schema.json
  PHENOTYPE_REVISION.schema.json
  CANONICAL_EVENT.schema.json
  ROLE_BINDING.schema.json
  GAP_SIGNAL.schema.json
  EVALUATION_RECEIPT.schema.json
  OUTBOX_OBLIGATION.schema.json
  EFFECT_RECEIPT.schema.json
/governance/
  DB_ROLE_MATRIX.yaml
  FORBIDDEN_TRANSITIONS.yaml
  PROMOTION_POLICY.yaml
  PROOF_REGISTRY.json
/runtime/
  sequencer/
  replay/
  task_bus/
  brain_gateway/
  context_compiler/
  effect_executor/
/observation/
  telemetry_schema/
  aggregators/
  detectors/
/development/
  mason/
/evidence/
  unit/
  integration/
  mutation/
  restore/
  provider_handoff/
  effects/
/releases/
  manifest.json
  SHA256SUMS

```

پیوست E - Procedure بازسازی برای دو سال بعد

Maintainer آینده نباید کار را با باز کردن Workflow قدیمی یا انتخاب جدیدترین Model شروع کند. ابتدا Architecture Version از Release Manifest مشخص می‌شود. بعد Schema, DB Role Matrix, Current و Canonical Head, Active Objective Contract, Active Phenotype Revision, Proof Registry و آخرین Restore Receipt بررسی می‌شوند. Replay Verification, CAS/Idempotency/Fencing Test, Mason Negative Test, Evaluator Attribution, Provider Handoff و Effect Reconciliation دوباره Verify می‌شوند. فقط بعد از هماهنگ شدن این Foundation ها Development جدید شروع می‌شود.

پیوست F - وضعیت Vocabulary باقی مانده از World 7

واژه‌های Non-extinction و Genome, Phenotype, Development, Genesis, Evolution, Sleep, Repair از History پروژه حذف نمی‌شوند. در Z0-A فقط معنای Operational که به Contract صریح نگاشت شود نگه داشته می‌شود. Phenotype یعنی Runtime Revision نسخه‌دار. Repair یعنی Governed Revision یا Recovery Transition. Sleep/Wake Operation های Lifecycle هستند. Genome/Evolution تا زمانی که Evidence تجربی مستقل وجود نداشته Population, Heritable Variation, External Fitness, Selection باشد Exploratory است. هیچ ادعای Biological Life وجود ندارد.

پیوست G - فایل‌های منبع Baseline

PDF های اصلی Baseline, Execution Roadmap و Cross-Review Closure بدون تغییر داخل بسته آرشیوی قرار می‌گیرند و به PDF نهایی ضمیمه می‌شوند. نسخه TXT/MD نیز متن استخراج شده را حفظ می‌کند.

معماری نهایی World 8 - خط مبنای Z0-A

سامانه عملیاتی پایدار، مستقل از ارائه دهنده، با مشاهده مستقل و توسعه پیشنهاد-محور

وضعیت: FINAL DESIGN BASELINE / NOT PRODUCTION

تاریخ تثبیت: 2026-08-24

دامنه: معماری هنجاری Z0-A؛ پیاده سازی، آزمون و ارتقای شواهد در رجیستری جداگانه دنبال می شود.

اصل مرکزی	مغز و رابط قابل تعویض اند؛ هویت، وضعیت کانیکال، اختیار، هدف و تاریخچه پذیرفته شده خارج از آنها می ماند.
هندسه نهایی	پنج Plane ثابت: Canonical Spine, Operational, Observation, Development/Mason, Evidence Governance.
مرز Mason	فقط Proposal؛ بدون حق تغییر مستقیم Canonical Objective, Hard Constraints, Authority, Identity, Canonical Spine یا Promote.
مرز ادعا	این سند «نهایی شدن طراحی Z0-A» را ثبت می کند، نه آمادگی Production، خودمختاری کامل یا تکامل خودکار.

1. حکم نهایی معماری

World 8 در Z0-A یک Runtime پایدار و قابل ممیزی برای موجودات/سازمان‌های دیجیتال است که شناخت، کانال و مدل را از مالکیت هویت و حقیقت جدا می‌کند. تصمیم نهایی پس از بازبینی متقاطع این است که هیچ سامانه توسعه‌گر، مدل زبانی، داشبورد، سیستم تله‌متری یا مخزن کد نباید بتواند خود را منبع حقیقت Runtime جا بزند.

قاعده پایه: هر تغییر معتبر باید از یک وضعیت کانیکال معلوم به یک وضعیت کانیکال جدید برسد، با Proposal مشخص، اختیار معتبر، CAS در زمان Commit-only، تاریخچه append-only، و Evidence متصل به همان Claim.

2. پنج Plane ثابت Z0-A

Plane	مالک چه چیزی است؟	ممنوعیت اصلی
1. Canonical Spine	/World/Entity identity, event history, head, version, checkpoints, authorizations, canonical revisions	مدل، Observation، UI، یا Mason حق نوشتن مستقیم در حقیقت کانیکال ندارند.
2. Operational	Entity/Role/RoleBinding, Task, ingress /Brain Gateway, context loading, outbox effects, connectors	کانیکال Provider/Session/Channel نباید به هویت کانیکال تبدیل شود.
3. Observation	Telemetry خام، normalization, aggregation, SLO evaluation, gap signals	Observation منبع Truth نیست و نباید توسط Mason انتخاب/دستکاری شود.
4. Development / Mason	ساخت Proposal برای Phenotype Revision بر اساس gap معتبر	خود-تغییری، self-approval، تغییر Hard Constraints/Authority/Identity/Spine و Promote مستقیم ممنوع.
5. Evidence Governance	Proof Registry, evaluator receipts approvals, promotion gates, audits, policy	PASS همسایه، Claim دیگر را ارتقا نمی‌دهد؛ evaluator باید مستقل و قابل انتساب باشد.

در نسخه قبلی، Cognition و Ingress ممکن بود به صورت Plane‌های جدا دیده شوند. در Z0-A آن‌ها سرویس‌ها و مرزهای درون Operational هستند تا توپولوژی هنجاری به پنج Plane کاهش یابد و مسئولیت‌ها شفاف‌تر شود.

3. مدل هویت: Role با Entity یکی نیست

Entity یک موجود/سازمان با هویت پایدار، lifecycle و state head مستقل است. Role یک جایگاه مسئولیت است. RoleBinding اتصال governed میان Role و Entity/Society/Executor است. بنابراین company-001 یک Entity/Society عملیاتی است، اما secretary-role یک Role است و از طریق role_binding فعال می‌شود.

نمونه	وضعیت مستقل؟	هویت مستقل؟	نشیء
company-001	بله	بله	Entity / Society
secretary-role	تعریف/نسخه دارد اما Entity نیست	خیر، جایگاه مسئولیت است	Role

-company-001 ↔ secretary role	بله، binding state دارد	خیر	RoleBinding
ChatGPT, Grok, API session	فقط provenance/runtime metadata	خیر	/Provider/Model Session

4. قرارداد هدف و نسخه فنوتیپ

4.1 Objective Contract - نسخه‌دار و کانیکال

هدف نباید در Prompt یا حافظه موقتی دفن شود. هر Objective Contract یک شیء نسخه‌دار است و حداقل شامل شناسه، نسخه، risk، authority ceiling، hard constraints، success metrics/SLO، objective، scope، adaptation surface، observation schema، class و promotion policy می‌شود.

```
objective_contract = {
  objective_contract_id,
  version,
  scope,
  objective,
  success_metrics_and_slos,
  hard_constraints,
  authority_ceiling,
  risk_class,
  observation_contract,
  allowed_adaptation_surface,
  promotion_policy
}
```

تغییر هدف، محدودیت سخت یا سقف اختیار یک «ویرایش عادی Mason» نیست؛ نیازمند مسیر governance /constitutional change جداگانه و مجوز صریح است.

4.2 Phenotype Revision - تغییر اجرایی نسخه‌دار

هر تغییر در نحوه عمل Runtime به شکل یک Phenotype Revision نسخه‌دار ثبت می‌شود: diff، parent revision، migration/rollback، پیش‌بینی اثر، معیار پذیرش، proposer، evidence refs، risk و status. نسخه قبلی پاک یا بازنویسی نمی‌شود.

```
phenotype_revision = {
  revision_id, parent_revision_id,
  objective_contract_version,
  change_set,
  migration_plan, rollback_plan,
  target_metrics, risk_class,
  evidence_refs,
  proposed_by, evaluated_by,
  status
}
```

5. Observation مستقل - قبل از Mason

Observation باید قبل از هر سامانه توسعه‌گر وجود داشته باشد. جریان هنجاری:

```

raw telemetry
-> normalization
-> aggregation/windowing
-> SLO / invariant evaluation
-> gap_signal
-> Evidence/Governance registry
-> Mason may consume the signal

```

Gap باید از Objective Contract و داده مشاهده شده مشتق شود. Mason حق ندارد خودش ورودی‌ها را گزینشی انتخاب کند، SLO را بازتعریف کند یا «مشکل» را صرفاً برای توجیه تغییر بسازد. برای تشخیص gap می‌توان از منطق deterministic استفاده کرد؛ در صورت نیاز classifier مستقل مجاز است، اما آن classifier نیز باید خارج از Mason، نسخه‌دار، قابل آزمون و قابل تعویض باشد.

خروجی Observation	فیلدهای حداقلی
gap_signal	gap_id, objective_contract_version, metric/invariant, window, expected, observed severity, provenance, detector_version, created_at
SLO breach	نام SLO, threshold, sampled values, aggregation method, breach duration, confidence quality flags
health signal	scope, signal type, state, evidence pointers, non-canonical marker

6. Mason - توسعه‌گر با دست بسته

Mason در Z0-A «عامل خودتکاملی» نیست. یک proposer محدود است که فقط پس از دریافت gap معتبر می‌تواند Candidate Revision بسازد.

- می‌تواند یک تغییر کوچک و قابل بازگشت پیشنهاد دهد.
- باید gap_id و Objective Contract version را مرجع کند.
- باید change-set, migration, rollback, expected impact و test plan ارائه کند.
- نمی‌تواند Objective, Hard Constraints, Authority, Identity یا Canonical Spine را تغییر دهد.
- نمی‌تواند evaluator خودش باشد، نتیجه ارزیابی را بنویسد یا Candidate را Promote کند.
- نمی‌تواند Observation data set را به‌دلخواه انتخاب کند تا Gap مصنوعی بسازد.

اصل کنترل: Mason may propose; Observation may measure; Evaluator may judge; Promotion authority may promote. هیچ نقش واحدی نباید هر چهار کار را انجام دهد.

7. Evaluator مستقل و هویت ارزیاب

Constraint پایگاه داده می‌تواند محدود کند «چه DB role ای اجازه نوشتن دارد»، اما به‌تنهایی ثابت نمی‌کند چه فرد/سرویس/نسخه کدی ارزیابی را انجام داده است. در Z0-A هر Evaluation Receipt باید هویت عملیاتی ارزیاب را به نتیجه bind کند.

فیلد	هدف
------	-----

هویت authenticated سرویس/انسان ارزیاب	evaluator_principal_id
مرز اختیار و credential reference؛ secret داخل receipt ذخیره نمی‌شود	evaluator_role / credential_ref
نسخه دقیق evaluator code/model/profile	evaluator_build_digest
اتصال نتیجه به ورودی دقیق	candidate_revision_id + objective_version
قابلیت تکرار و تشخیص drift	test_suite_digest / environment
خروجی قابل ممیزی	result + metrics + evidence refs
تقویت انتساب؛ جایگزین کنترل دسترسی نیست	attestation/signature where available

Evaluator و Mason باید credential و DB role جدا داشته باشند. Promotion نیز capability جداگانه دارد. این Separation of Duties باید هم در سطح مدل داده و هم با PostgreSQL ROLE/GRANT/REVOKE اعمال و آزمون شود.

8. Canonical Spine - قرارداد commit نهایی

Spine از primitive های شناخته شده استفاده می‌کند و ادعای اختراع CAS، event sourcing، outbox یا fencing ندارد. هدف آن ایجاد یک مرز continuity قابل ممیزی است.

8.1 Commit-time CAS

CAS باید در همان تراکنشی که head را تغییر می‌دهد و در «زمان commit» بررسی شود. چک pre-read در application کافی نیست. stale expected head برای write جدید باید fail-closed شود.

8.2 Idempotency با collision handling

Idempotency key به «bind intent» می‌شود. retry با همان semantics می‌تواند prior result را برگرداند؛ reuse همان key با event type/payload متفاوت خطاست و نباید silently overwrite کند.

8.3 Atomic append/head/outbox

برای عملیاتی که اثر خارجی برنامه‌ریزی می‌کند، create و append event، advance head، update projections، outbox obligation باید در یک transaction اتمیک انجام شوند.

8.4 Transactional Outbox + Effect Receipts

ارسال ایمیل، پیام، ثبت CRM یا هر side effect باید obligation مستقل داشته باشد. audit ها attempt می‌شوند و موفقیت نهایی با receipt و id effect provider-side در صورت وجود settle می‌شود. 8 World فقط -effectively once را در محدوده key/connector/receipt مشخص هدف می‌گیرد؛ universal exactly-once ممنوع است.

8.5 Sequencer lease + fencing

یک active sequencer lease با fencing token لازم است. sequencer جابه‌جا یا expired نباید با token قدیمی commit کند، حتی اگر process قدیمی هنوز زنده باشد.

8.6 Append-only در DB

رویدادهای committed نباید فقط با convention غیرقابل ویرایش باشند؛ DB باید UPDATE/DELETE/TRUNCATE مسیر Runtime را با role/trigger/privilege کنترل کند. اصلاح گذشته، رویداد جدید است.

Checkpoint, replay, clean-host restore 8.7

Replay باید قبل از materialization زنجیره، duplicate IDs، version sequence، hash را verify کند. Checkpoint به head/version bind می‌شود. Restore معتبر فقط وقتی است که یک محیط پاک از artifacts durable بازسازی، verify و سپس یک write governed جدید را با lease تازه قبول کند.

Context و Provider-independent identity 9

World/Entity/Role/Task، provenance، channel و Provider، model، session، thread identity از آن‌ها مشتق نمی‌شود. تعویض Provider باید همان canonical IDs و Task state را حفظ کند. Context loading نیز به صورت scoped انجام می‌شود:

لایه	محتوا
K0	World kernel و invariant های لازم
K1	Entity/Society objective و shared constraints
K2	Role contract، permissions، output contract
K3	Task جاری، state، artifacts، approvals
K4	حداقل evidence/memory مرتبط با provenance

10. جریان کامل Runtime و توسعه

External input

- > Operational ingress/auth/route
- > scoped context + replaceable cognition
- > proposal/result
- > policy/authority evaluation
- > Canonical Spine commit (CAS + lease + idempotency)
- > task/state projection + optional outbox
- > effect attempt -> receipt
- > Observation telemetry
- > aggregation / SLO evaluation
- > gap_signal
- > Mason proposal (optional)
- > independent evaluation
- > Governance promotion decision
- > canonical Phenotype Revision activation

Evidence / Governance 11

هر Claim باید status، predicate، falsifier، required evidence، evidence references مستقل داشته باشد. تعداد تست، جای کیفیت شواهد را نمی‌گیرد. OPEN یعنی شاهد لازم وجود ندارد. Evidence سطح پایین‌تر نمی‌تواند ادعای سطح بالاتر را خودکار ارتقا دهد.

سطح	معنی Z0-A
-----	-----------

E0	تعریف/ایده بدون اجرای شاهد
E1	قرارداد/schema/privilege مشخص و قابل ابطال
E2	اجرای local/disposable و mutation-sensitive
E3	اجرای persistent target یا integration واقعی
E4	production-like: restore/security/load/chaos/independent credentials/runbooks
E5	longitudinal operation با SLO و failure envelope از پیش تعیین شده

12. امنیت و Separation of Duties

می تواند	نمی تواند	Capability/Role
Human Root / Promotion Authority	approve/revoke promotion یا policy حساس	rewrite history؛ جایگزین همه کنترل های امنیتی شود
Sequencer	commit under lease/fencing/CAS	bypass revocation؛ self-approve
Operational Executor	Task work و request effect	mint authority یا تغییر identity
Observer	read telemetry و produce signed attributed signals	canonical mutation یا define objective
Mason	propose phenotype changes	promote، self-evaluate، تغییر hard constraints
Evaluator	evaluate candidate	propose همان candidate یا promote آن
Effect Executor	settle authorized outbox obligations	create its own authorization

13. قواعد Rollback و Promotion

1. Gap معتبر توسط Observation ثبت شود.
2. Mason یا انسان Candidate Revision بسازد.
3. Candidate در shadow/replay/test profile مشخص ارزیابی شود.
4. Evaluator Receipt با هویت، build digest و evidence ثبت شود.
5. Policy/Promotion gate بررسی کند hard constraints و authority دست نخورده اند.
6. Promotion Authority تصمیم بدهد.
7. فعال سازی revision با expected current revision/head و CAS کانیکال commit شود.
8. در regression یا rollback، breach با event جدید به revision قبلی/known-good انجام شود.

14. Non-claims و خطوط قرمز

- این معماری ادعای آگاهی، حیات زیستی، AGI یا شخصیت حقوقی ندارد.
- Mason در Z0-A «خودتکاملی» اثبات شده نیست.

- Hash chain برابر secure storage در برابر superuser نیست.
- DB constraints به‌تنهایی هویت evaluator را اثبات نمی‌کنند.
- Human approval به‌تنهایی security boundary کافی نیست.
- External effect universal exactly-once ادعا نمی‌شود.
- Production readiness تا بسته‌شدن security/load، persistent restore، credential separation، chaos و SLOها OPEN است.

معيار PASS	Gate
پنج Plane در schema/API/DB roles مرزبندی شده؛ bypass test fail-closed.	Z0A-G1 Plane isolation
Entity و Role جدا؛ RoleBinding portable؛ provider field در canonical identity لازم نیست.	Z0A-G2 Identity/Role split
Objective Contract و Phenotype Revision immutable/versioned با CAS .promotion	Z0A-G3 Objective/Revision versioning
raw→aggregate→SLO→gap مستقل از Mason؛ provenance کامل.	Z0A-G4 Observation independence
تست منفی ثابت کند Mason نمی‌تواند Objective/Authority/Identity/Spine را عوض یا Promote کند.	Z0A-G5 Mason confinement
receipt به bind principal/build/input/output شده و evaluator credential از Mason جداست.	Z0A-G6 Evaluator identity
CAS/idempotency/append/head/outbox atomic stale/fencing/collision؛ tests PASS.	Z0A-G7 Spine atomicity
clean-host restore + fresh post-restore governed write + new fencing .token PASS	Z0A-G8 Recovery
fail-closed و authorized outbox→attempt→receipt. duplicate-effect test .ambiguity	Z0A-G9 Effects

نامزد v0.1/v0.1.1 همچنان شاهد عملیاتی مهمی برای company-001، world-001، نقش‌ها، Task Bus و زنجیرهٔ persistent است. Z0-A این lineage را با چهار اصلاح معماری اصلی می‌بندد: تثبیت پنج Plane، استقلال Observation از Mason، نسخه‌دار شدن Objective/Phenotype Revision، و اثبات‌پذیر کردن هویت evaluator فراتر از DB constraint. هیچ شاهد قدیمی به‌صورت خودکار این الزامات جدید را PASS نمی‌کند؛ آن‌ها Gate‌های Z0-A هستند.

WORLD 8 / Z0-A

نقشه اجرایی و Gate های پذیرش Z0-A

از تثبیت قراردادهای Observation مستقل، Mason محدود، ارزیابی مستقل و Restore

وضعیت: EXECUTION ROADMAP / FINAL DESIGN BASELINE

اصل: هیچ مرحله با گذشت زمان PASS نمی شود؛ فقط Evidence می تواند Gate را ببندد.

1. ترتیب اجرا

مرحله	خروجی	Gate اصلی
Z0A-0 Freeze contracts	Plane boundaries, schemas, Objective/Revision contracts, role model	Contract lint + schema validation + change-control freeze
Z0A-1 Spine hardening	/commit-time CAS, intent idempotency, atomic append head/outbox, lease/fencing	race/adversarial/mutation gate
Z0A-2 Entity/Role separation	portable binding و Entity, Role, RoleBinding tables/APIs	wrong-role/provider-capture negative tests
& Z0A-3 Objective Revision	Phenotype Revision و versioned Objective Contract	immutability + CAS promotion + rollback
Z0A-4 Observation	raw telemetry→aggregate→SLO→gap_signal	provenance, detector version, Mason isolation
Z0A-5 Mason proposer	Candidate Revision فقط از gap معتبر	forbidden-change negative suite
Z0A-6 Independent evaluation	/evaluation receipts + separate evaluator principal build	SoD + identity binding tests
/Z0A-7 Promotion rollback	promotion gate + canonical activation event	;self-approval impossible stale revision rejected
Z0A-8 Recovery	checkpoint/replay/clean-host restore	+ fresh post-restore write new fencing token
Z0A-9 Operational proof	provider handoff + real effect receipt + SLO monitoring	E3/E4 claim-specific gates

2. Z0A-0 - Freeze قراردادهای پایه

- ثبت پنج Plane و owner هر object.
- تعریف schemaهای Objective Contract, Phenotype Revision, gap_signal, Evaluation Receipt.
- ثبت forbidden transitions.
- تعریف DB roles: runtime_sequencer, observer, mason_proposer, evaluator, effect_executor, promotion_authority.
- تعریف GRANT/REVOKE ماتریس و threat assumptions.

3. Z0A-1 - Spine

- CAS داخل transaction نهایی.
- idempotency key + semantic hash collision rejection.
- atomic event/head/projection/outbox.

- lease expiry/rotation + fencing
- append-only at DB level
- checkpoint verification and replay-integrity checks

آزمون‌های اجباری

Expected	Test
فقط اولی commit؛ دومی stale reject	دو writer با expected_head یکسان
deduplicate/replay prior result	retry همان key و همان intent
partial write هیچ؛ IDEMPOTENCY_COLLISION	همان key با payload متفاوت
reject	stale fencing token
all-or-nothing	event commit + outbox fault injection
runtime role denied	UPDATE/DELETE committed_events

Z0A-2 - Entity/Role/Binding .4

company-001 به عنوان Entity/Society باقی می‌ماند. sales-role، secretary-role و accountant-role جایگاه هستند. Binding ها executor/provider-neutral می‌مانند.

- Role name authority mint نمی‌کند.
- Task assignment با RoleBinding فعال و scope مجاز تطبیق داده می‌شود.
- Provider switch هیچ canonical ID را تغییر نمی‌دهد.

Z0A-3 - Objective / Revision .5

- Objective Contract immutable per version
- rollback-ready و Phenotype Revision parent-linked
- Mason فقط allowed_adaptation_surface را لمس می‌کند.
- hard constraints و authority ceiling خارج از مسیر Mason هستند.

Z0A-4 - Observation .6

- Telemetry loss نباید replay را مختل کند.
- Aggregation window و detector version ثبت شود.
- gap_signal باید provenance و objective version داشته باشد.
- Gap detector مستقل از Mason credential و code path باشد.

Evaluator و Z0A-5/6 - Mason .7

Evaluator	Mason	موضوع
-----------	-------	-------

می‌تواند صحت مرجع gap را بررسی کند	مصرف می‌کند	Gap
ارزیابی می‌کند	می‌سازد	Candidate
باید verify کند تغییر نکرده‌اند	حق تغییر ندارد	Objective/Hard Constraints
ندارد	ندارد	Promotion
identity/build digest با evaluation receipt	proposal receipt	Receipt

Z0A-7 - Promotion .8

Promotion Authority تنها بعد از receipt معتبر و policy gate می‌تواند Candidate را فعال کند. activation باید روی CAS current phenotype revision/head داشته باشد. هر rollback یک canonical event جدید است.

Z0A-8 - Restore .9

1. Export durable recovery bundle
2. Start clean environment with different system identity
3. Restore schema/data
4. Verify chain/checkpoint/head/revision/objective
5. Recreate separated DB roles/credentials
6. Acquire fresh lease with higher fencing token
7. Execute one fresh governed write
8. Record restore receipt, RTO/RPO, operator steps

Z0A-9 - Operational evidence .10

- ChatGPT→Grok handoff on same Task without identity fork
- One real low-risk external effect with authorization/outbox/attempt/receipt
- Observation detects a predefined SLO breach
- Mason proposes a reversible low-risk improvement from that gap
- Independent evaluator judges it; separate promotion decision activates or rejects

Release rule .11

A .Z0-A is complete only when architecture and executable boundary agree
document claiming five planes while runtime still allows Mason to select its own telemetry
or promote itself is a FAIL, regardless of test count

WORLD 8 / Z0-A

ثبت نهایی تصمیمات Cross-Review

Closure Record برای معماری شاخه‌ای Z0-A

وضعیت: DESIGN DECISIONS CLOSED FOR Z0-A

منبع: جمع‌بندی بازبینی متقاطع؛ این سند نقل‌قول کلمه‌به‌کلمه داورها نیست، بلکه تصمیمات نهایی معماری را ثبت می‌کند.

1. تصمیمات بسته شده

موضوع	تصمیم نهایی	علت معماری
تعداد Plane	پنج Plane ثابت	کاهش ambiguity و جلوگیری از مالکیت Truth توسط interface/model/telemetry
Entity vs Role	کاملاً جدا	Role مسئولیت است؛ Entity هویت/state مستقل دارد
company-001	Entity/Society	دارای هویت عملیاتی و history مستقل
secretary-role	Role با role_binding	قابل تعویض بودن holder/provider بدون تغییر Role identity
Objective	Objective Contract نسخه دار	هدف نباید prompt-local یا قابل دستکاری Mason باشد
Runtime changes	Phenotype Revision نسخه دار	CAS promotion و rollback, audit
Mason	Proposal-only	جلوگیری از self-promotion و self-modification
Observation	مستقل و قبل از Mason	جلوگیری از cherry-picking data و self-defined gaps
Canonical Spine	commit-time CAS + idempotency collision atomic outbox + fencing +	fail-closed concurrency و continuity
Append-only	DB-enforced	policy convention به تنهایی کافی نیست
Restore	checkpoint/replay/clean-host + fresh write	پایداری هویت بدون recovery واقعی ناقص است
SoD	DB roles/GRANT + credential separation	Human approval یا app-level checks کافی نیستند
Evaluator identity	receipt باید principal/build/input/output را bind کند	DB constraint فقط capability را نشان می دهد، نه هویت واقعی evaluator

2. موارد رد شده برای Z0-A

- Mason که خودش telemetry را انتخاب کند و gap بسازد.
- Mason که Objective یا Hard Constraints را rewrite کند.
- Promotion مستقیم از Development Plane.
- Role به عنوان Entity مستقل فقط چون اسم مسئولیت دارد.
- Hash chain به عنوان secure storage کافی.
- Human click به عنوان تنها security boundary برای effect پرخطر.
- DB constraint به عنوان «اثبات هویت evaluator».
- Universal exactly-once.

- هرگونه ادعای autonomous evolution بدون evidence مستقل.

3. مرز Evidence

نامزد v0.1.1 و شواهد E2/E3 موجود، lineage عملیاتی World 8 را نشان می‌دهند. ولی هر Claim جدید Z0-A باید Gate خودش را پاس کند. به‌خصوص evaluator identity، Observation independence، Mason و confinement نیازمند آزمون‌های جدید هستند.

4. اصل نهایی Separation of Roles

Observation measures
Mason proposes
Evaluator judges
Promotion Authority promotes
Canonical Spine records

اگر یک actor بیش از یکی از این نقش‌ها را برای همان change بدون کنترل مستقل انجام دهد، معماری Z0-A نقض شده است.

5. نتیجه

Cross-review به یک معماری «کم‌تر شاعرانه، سخت‌گیرتر و قابل ابطال‌تر» ختم شد: توسعه همچنان ممکن است، اما خود توسعه‌گر مالک هدف، داده قضاوت، ارزیابی و ارتقا نیست. این تصمیم، Mason را از «عامل خودتکاملی» به یک مهندس پیشنهاددهنده در یک خط تولید governed تبدیل می‌کند.